

# Deploy da demonstração na Vercel

A demonstração publicada usa dois projetos na Vercel, apontando para o mesmo repositório, e um PostgreSQL gerenciado (Neon, instalado pelo marketplace da própria Vercel).

```
Navegador —> frontend (Vercel, Vite/React) —> backend (Vercel, Django como função Python) —> PostgreSQL (Neon)
```

A Vercel detecta projetos Django sem configuração extra desde abril de 2026: encontra o `manage.py`, lê o `WSGI_APPLICATION`, roda o `collectstatic` e publica a aplicação como função. O backend tem só três arquivos específicos da Vercel:

- `pyproject.toml` : dependências e o script de build;
- `build_vercel.py` : aplica as migrações e monta o cenário de demonstração a cada deploy (idempotente), usando a conexão direta do banco ( `DATABASE_URL_UNPOOLED` ), e não a do pool;
- `vercel.json` : tempo máximo de 60 segundos para a função.

## Por que Vercel e não EC2 para a demonstração

| Critério                           | Vercel + Neon                                         | EC2                                       |
|------------------------------------|-------------------------------------------------------|-------------------------------------------|
| Custo para uma demonstração        | Plano gratuito cobre                                  | Instância ligada o tempo todo             |
| Tempo até o primeiro link          | Minutos, a partir do GitHub                           | Configurar servidor, proxy, TLS, processo |
| Preview por commit                 | Automático                                            | Manual                                    |
| Adequação à produção deste sistema | Parcial: sem disco persistente e sem processos longos | Boa: workers de extração e RPA            |

Para produção a proposta mantém EC2 para API e workers (extração, RPA com Playwright, Celery), e Vercel para o frontend. Ver `01-proposta.md`, seção 5.2. O protótipo foi adaptado para rodar nos dois cenários sem mudança de código, só com variáveis de ambiente.

## Adaptações feitas para o ambiente serverless

- **Extração em memória:** os extratores recebem bytes, não caminho de arquivo.
- **Originais no banco:** com `ORIGINAIS_NO_BANCO=1` (ligado sozinho quando `VERCEL=1`), o PDF original também é gravado no PostgreSQL e servido por `/api/documentos/{id}/original/`. Na Vercel o disco só permite escrita em `/tmp`, que não persiste. Em produção o lugar do original é o S3.
- **Amostras pela API:** `/api/amostras/` lista e serve os PDFs fictícios, para quem testa pelo link não precisar baixar o repositório.
- **Reinício da demonstração:** `POST /api/demo/reiniciar/` recria o cenário. Só funciona com `DEMO_PERMITE_REINICIAR=1`.

## Passo a passo

### 1. Backend e banco

1. *Add New Project*, importe o repositório e defina **Root Directory** = `backend`.
2. Antes do primeiro deploy, em *Storage*, crie um banco Neon (PostgreSQL) e conecte ao projeto. A integração cria `DATABASE_URL` e `DATABASE_URL_UNPOOLED`.
3. Variáveis de ambiente:

| Variável                                                          | Valor                                                                                            |
|-------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| <code>DATABASE_URL</code> ,<br><code>DATABASE_URL_UNPOOLED</code> | vêm da integração do Neon                                                                        |
| <code>DJANGO_SECRET_KEY</code>                                    | uma chave longa e aleatória                                                                      |
| <code>DJANGO_DEBUG</code>                                         | <code>0</code>                                                                                   |
| <code>DJANGO_ALLOWED_HOSTS</code>                                 | <code>.vercel.app</code>                                                                         |
| <code>CORS_ALLOWED_ORIGINS</code>                                 | URL do frontend (preencha depois do passo 2), ex.: <code>https://cotador-dados.vercel.app</code> |
| <code>CORS_ALLOWED_ORIGIN_REGEX</code>                            | opcional, para previews: <code>^&lt;https://cotador-dados-&gt;.*\.vercel\.app\$</code>           |
| <code>DEMO_PERMITE_REINICIAR</code>                               | <code>1</code>                                                                                   |

4. Faça o deploy. No log do build devem aparecer `Applying catalogo.0003...` e `Pronto.`
5. Confira `https://<backend>.vercel.app/api/`: deve responder `{"ok": true, ...}`.

**Plano B, se o build não migrar** (por exemplo, se a Vercel ignorar o `pyproject.toml`): rode no seu computador, com a URL direta copiada do painel do Neon:

```
cd backend
pip install -r requirements.txt
DATABASE_URL="postgresql://...?sslmode=require" python build_vercel.py
```

A migração `0003` cria a extensão `btree_gist`. Se o usuário do banco não puder criar extensões, rode `CREATE EXTENSION btree_gist;` no console SQL do Neon e faça o deploy de novo.

### 2. Frontend

1. Novo projeto no mesmo repositório, **Root Directory** = `frontend` (a Vercel detecta Vite).
2. Variável `VITE_API_URL` = URL do backend, sem barra no final.
3. Deploy.
4. Volte ao projeto do backend, preencha `CORS_ALLOWED_ORIGINS` com a URL do frontend e faça *Redeploy* (variável nova só vale depois de um novo deploy).

## Conferência depois do deploy

Rode esta lista antes de enviar o link. Um link quebrado é pior do que nenhum link.

- [ ] `/api/` responde `ok: true`.
- [ ] `/api/amostras/` lista os três PDFs (confirma que a pasta `amostras` foi para o pacote da função).
- [ ] A tela abre sem a faixa vermelha "Não foi possível falar com a API" (confirma CORS).
- [ ] O roteiro de `04-roteiro-demonstracao.md` roda do início ao fim.
- [ ] "Ver documento" na cotação abre o PDF (confirma originais no banco).
- [ ] "Reiniciar demonstração" volta o cenário ao estado inicial.
- [ ] Abrir o link numa janela anônima e no celular.

## Problemas prováveis

| Sintoma                                                         | Causa provável                                     | Correção                                                                                           |
|-----------------------------------------------------------------|----------------------------------------------------|----------------------------------------------------------------------------------------------------|
| Faixa vermelha "Não foi possível falar com a API"               | CORS ou <code>VITE_API_URL</code> errado           | Conferir as duas variáveis; o frontend precisa de novo deploy após mudar <code>VITE_API_URL</code> |
| <code>DisallowedHost</code>                                     | <code>DJANGO_ALLOWED_HOSTS</code> sem o domínio    | Usar <code>.vercel.app</code>                                                                      |
| Lista de amostras vazia                                         | Pasta <code>backend/amostras</code> fora do pacote | Confirmar que os PDFs estão versionados no Git                                                     |
| Erro na migração <code>0003</code>                              | Sem permissão para criar extensão                  | Criar <code>btree_gist</code> pelo console do Neon                                                 |
| Build falha em <code>build_vercel.py</code> com erro de conexão | Banco não conectado ao projeto antes do deploy     | Conectar o Neon em <i>Storage</i> e fazer <i>Redeploy</i>                                          |
| Primeira requisição lenta                                       | Função fria                                        | Normal; as seguintes são rápidas                                                                   |